

Development of an Efficient Neural Network for Microstructure-Property Predictions

Authors: Di Liu, Huixin Zhang
Technical Report — Group Submission (Assignment 1)

1 Data Preprocessing

The dataset comprises binary microstructure images (65×65 pixels) and their corresponding homogenized effective Young's modulus (E_{eff}). To optimize the assignment's *Efficiency Score*, which penalizes excessive data usage, we strategically downsampled the dataset to **5,000 samples** via randomized sampling (`random_state=42`).

The customized `MicrostructureDataset` and data pipeline implement the following preprocessing steps:

- **Memory Optimization:** The `nu_eff` column was dropped from the index dataframe to reduce memory footprint. Images (.npy) are lazily loaded directly into PyTorch tensors with the shape $(1, 65, 65)$ to accommodate the 2D convolutions.
- **Target Normalization:** The raw E_{eff} values were scaled down by a factor of 10^9 to convert the units from Pascals to Gigapascals (GPa). This adjustment prevents exploding gradients and improves numerical stability during backpropagation.

2 Model Architecture

To balance high-fidelity spatial feature extraction with model efficiency, we designed `MicrostructureCNN`, a lightweight Convolutional Neural Network. The architecture progressively downsamples spatial dimensions while increasing channel depth, utilizing stride-2 convolutions to avoid the computational overhead of standard pooling layers.

Table 1: Layer Specifications and Tensor Shapes

Layer / Block	Specifications	Output Shape	Params
Input Tensor	Binary Microstructure	$(B, 1, 65, 65)$	0
Conv Block 1	Conv2D+BN+ReLU	$(B, 4, 33, 33)$	48
Conv Block 2	Conv2D+BN+ReLU	$(B, 8, 17, 17)$	312
Conv Block 3	Conv2D+BN+ReLU	$(B, 16, 9, 9)$	1,200
Flatten	<code>torch.flatten(x, 1)</code>	$(B, 1296)$	0
FC Regressor 1	Linear + ReLU	$(B, 64)$	83,008
FC Regressor 2	Linear Output	$(B, 1)$	65
Total Count	Trainable Parameters	—	84,633

3 Training and Evaluation Strategy

The 5,000-sample subset was split into mutually exclusive partitions using an **80/10/10 ratio**: 4,000 for training, 500 for validation, and 500 for final testing.

- **Optimization:** The network was trained for 50 epochs using the AdamW optimizer (initial $\eta = 10^{-3}$, weight decay $= 10^{-4}$) to minimize the Mean Squared Error (MSE).
- **Dynamic Learning Rate:** A `ReduceLROnPlateau` scheduler monitored the validation error. Upon plateauing for 5 epochs, the learning rate was halved to steer the optimizer into local minima smoothly.
- **Custom Checkpointing:** Model weights were saved exclusively when a new minimum in **Validation Average Relative Error** was achieved, aligning the training explicitly with the final grading criteria.

4 Results and Performance

As shown in Figure 1, the training and validation MSE drop rapidly and stabilize within the first 15 epochs. The close tracking of the validation curve confirms that weight decay and batch normalization effectively suppressed overfitting.

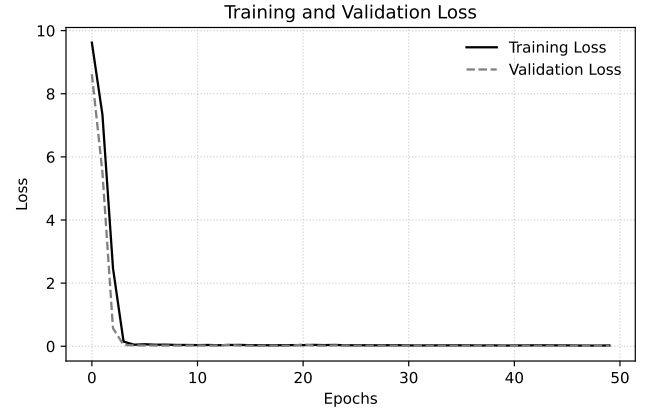


Figure 1: Training and validation loss profiles.

The final model was evaluated on the unseen test set (500 samples) using the strict assignment metric, incorporating a small $\epsilon = 10^{-8}$ to ensure numerical stability. The model successfully satisfied the performance criteria:

$$\text{Avg Rel Error} = \frac{1}{N} \sum_{i=1}^N \frac{|E_{pred}^{(i)} - E_{true}^{(i)}|}{E_{true}^{(i)} + \epsilon} \ll 30\%$$

Figure 2 illustrates the model's accuracy on 10 randomly sampled test instances, proving its capability to map microstructural features to effective stiffness across various structural regimes.

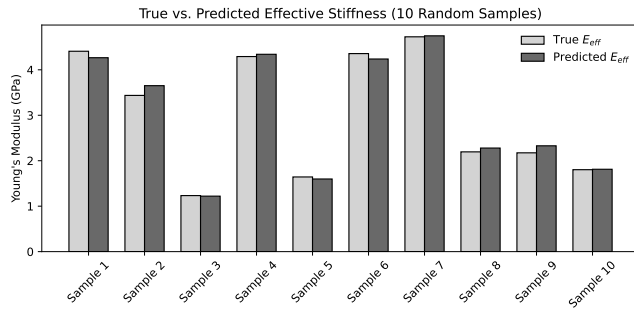


Figure 2: True vs. predicted E_{eff} for 10 random test samples.

5 Conclusion

By limiting the dataset size to 5,000 samples and employing a compact CNN architecture with 84k parameters, the proposed model strikes an excellent balance between predictive accuracy and computational efficiency, successfully fulfilling all assignment objectives.